

Задание 1

Цель: Добавить конструкторы, получающие массив FunctionPoint.

```
public ArrayTabulatedFunction(FunctionPoint[] points) {
    if (points == null || points.length < 2) {
        throw new IllegalArgumentException("Массив должен содержать не менее 2
элементов");
    }
    for (int i = 1; i < points.length; i++) {
        if (points[i].getX() <= points[i-1].getX()) {
            throw new IllegalArgumentException("Точки должны быть упорядочены по
возрастанию x");
        }
    }
    this.points = new FunctionPoint[points.length];
    this.pointsCount = points.length;
    for (int i = 0; i < points.length; i++) {
        this.points[i] = new FunctionPoint(points[i]);
    }
}
```

Аналогичный конструктор добавлен в LinkedListTabulatedFunction.

```
public LinkedListTabulatedFunction(FunctionPoint[] points) {
    if (points==null||points.length<2) {
        throw new IllegalArgumentException("Массив должен содержать не менее 2
элементов");
    }
    for (int i=1;i<points.length;i++) {
        if (points[i].getX()<=points[i-1].getX()) {
            throw new IllegalArgumentException("Точки должны быть упорядочены по
возрастанию x");
        }
    }
    initList();
    for (int i=0;i<points.length;i++) {
        FunctionNode node=addNodeToTail();
        node.data = new FunctionPoint(points[i]);
    }
}
```

Задание 2

Цель: Создать интерфейс Function и реорганизовать TabulatedFunction.

```
package functions;
public interface Function {
    double getLeftDomainBorder();
    double getRightDomainBorder();
    double getFunctionValue(double x);
}
```

```
package functions;
public interface TabulatedFunction extends Function {
    int getPointsCount();
    FunctionPoint getPoint(int index) throws FunctionPointIndexOutOfBoundsException;
    //... остальное
}
```

Задание 3

Создать пакет functions.basic с различными функциями.

```
package functions.basic;
import functions.Function;
```

```

public class Exp implements Function {

    @Override
    public double getLeftDomainBorder() {
        return Double.NEGATIVE_INFINITY;
    }

    @Override
    public double getRightDomainBorder() {
        return Double.POSITIVE_INFINITY;
    }

    @Override
    public double getFunctionValue(double x) {
        return Math.exp(x);
    }

}

```

```

package functions.basic;

import functions.Function;

// Класс логарифмической функции с заданным основанием

public class Log implements Function {

    private double base;
    // основание логарифма (должно быть > 0 и != 1)
    public Log(double base) {
        if (base <= 0 || base == 1) {
            throw new IllegalArgumentException("Основание логарифма должно быть положительным
и не равным 1");
        }
        this.base = base;
    }

    @Override
    public double getLeftDomainBorder() {
        return 0; // определен на (0, +infinity)
    }

    @Override
    public double getRightDomainBorder() {
        return Double.POSITIVE_INFINITY;
    }

    @Override
    public double getFunctionValue(double x) {
        if (x <= 0) {
            return Double.NaN;
        }
        // log_base(x) = ln(x) / ln(base)
        return Math.log(x) / Math.log(base);
    }

}

```

```

package functions.basic;

// Класс функции синуса;

public class Sin extends TrigonometricFunction {

    @Override
    public double getFunctionValue(double x) {
        return Math.sin(x);
    }

}

```

Идентично – cos, tan

```

package functions.basic;

import functions.Function;

// Базовый класс для тригонометрических функций

```

```

public abstract class TrigonometricFunction implements Function {
    /*
    Имело бы смысл сделать RealDomainFunction с 00 +-infinity,
    а уже от него TrigonometricFunction включающий RealDomainFunction
    с тригонометрическими особенностями, тк экспонента так же могла бы
    включать RealDomainFunction не нарушая математического смысла
    */

    @Override
    public double getLeftDomainBorder() {
        return Double.NEGATIVE_INFINITY;
    }

    @Override
    public double getRightDomainBorder() {
        return Double.POSITIVE_INFINITY;
    }
}

```

Задание 4

Цель: Создать пакет functions.meta для комбинирования функций.

```

package functions.meta;

import functions.Function;

public class Sum implements Function {

    private Function f1;
    private Function f2;

    public Sum(Function f1, Function f2) {
        this.f1 = f1;
        this.f2 = f2;
    }

    @Override
    public double getLeftDomainBorder() {
        // берем самое правое значение
        return Math.max(f1.getLeftDomainBorder(), f2.getLeftDomainBorder());
    }

    @Override
    public double getRightDomainBorder() {
        // берем самое левое значение
        return Math.min(f1.getRightDomainBorder(), f2.getRightDomainBorder());
    }

    @Override
    public double getFunctionValue(double x) {
        return f1.getFunctionValue(x) + f2.getFunctionValue(x);
    }
}

```

```

package functions.meta;

import functions.Function;

// Класс функции, являющейся композицией двух функций
// f(g(x))

public class Composition implements Function {

    private Function outer; // внешняя функция f
    private Function inner; // внутренняя функция g

    public Composition(Function outer, Function inner) {
        this.outer = outer;
        this.inner = inner;
    }

    @Override
    public double getLeftDomainBorder() {
        return inner.getLeftDomainBorder();
    }
}

```

```

@Override
public double getRightDomainBorder() {
    return inner.getRightDomainBorder();
}

@Override
public double getFunctionValue(double x) {
    return outer.getFunctionValue(inner.getFunctionValue(x));
}
}

```

Аналогично реализованы: Mult, Power, Scale, Shift.

Задание 5

Цель: Создать утилитный класс Functions.

```

package functions;

import functions.meta.*;

public final class Functions {

    // запрет создания объектов вне класса
    private Functions() {
        throw new UnsupportedOperationException("Нельзя создать объект класса Functions");
    }

    public static Function shift(Function f, double shiftX, double shiftY) {
        return new Shift(f, shiftX, shiftY);
    }

    // ...и тд
}

```

Задания 6-7

Цель: Создать класс TabulatedFunctions с методами табулирования и ввода-вывода.

```

package functions;

import java.io.*;

public final class TabulatedFunctions {

    private TabulatedFunctions() {
        throw new UnsupportedOperationException("");
    }

    public static TabulatedFunction tabulate(Function function, double leftX, double rightX, int
pointsCount) {
        if (leftX < function.getLeftDomainBorder() || rightX > function.getRightDomainBorder()) {
            throw new IllegalArgumentException("");
        }

        if (leftX >= rightX) {
            throw new IllegalArgumentException(" ");
        }

        if (pointsCount < 2) {
            throw new IllegalArgumentException("");
        }

        double[] values = new double[pointsCount];
        double step = (rightX - leftX) / (pointsCount - 1);

        for (int i = 0; i < pointsCount; i++) {
            values[i] = function.getFunctionValue(leftX + i * step);
        }

        return new ArrayTabulatedFunction(leftX, rightX, values);
    }
}

```

```

        public static void outputTabulatedFunction(TabulatedFunction function, OutputStream out) throws
IOException {
            DataOutputStream dataOut = new DataOutputStream(out);

            int count = function.getPointsCount();
            dataOut.writeInt(count);

            for (int i = 0; i < count; i++) {
                dataOut.writeDouble(function.getPointX(i));
                dataOut.writeDouble(function.getPointY(i));
            }

            dataOut.flush();
            /*
            поток НЕ закрывает - это задача вызывающего кода,
            который открыл поток и знает, нужно ли его дальше применять
            */
        }

        public static TabulatedFunction inputTabulatedFunction(InputStream in) throws IOException {
            DataInputStream dataIn = new DataInputStream(in);

            int count = dataIn.readInt();

            if (count < 2) {
                throw new IOException("'" + count);
            }

            FunctionPoint[] points = new FunctionPoint[count];

            for (int i = 0; i < count; i++) {
                double x = dataIn.readDouble();
                double y = dataIn.readDouble();
                points[i] = new FunctionPoint(x, y);
            }

            return new ArrayTabulatedFunction(points);
        }

        public static void writeTabulatedFunction(TabulatedFunction function, Writer out) throws
IOException {
            PrintWriter writer = new PrintWriter(out);

            int count = function.getPointsCount();
            writer.print(count);

            for (int i = 0; i < count; i++) {
                writer.print(" " + function.getPointX(i));
                writer.print(" " + function.getPointY(i));
            }

            writer.flush();
        }

        public static TabulatedFunction readTabulatedFunction(Reader in) throws IOException {
            StreamTokenizer tokenizer = new StreamTokenizer(in);

            tokenizer.nextToken();
            int count = (int) tokenizer.nval;

            FunctionPoint[] points = new FunctionPoint[count];

            for (int i = 0; i < count; i++) {
                tokenizer.nextToken();
                double x = tokenizer.nval;

                tokenizer.nextToken();
                double y = tokenizer.nval;

                points[i] = new FunctionPoint(x, y);
            }

            return new ArrayTabulatedFunction(points);
        }
    }

```

```
}
```

Задание 8

Тестирование всех компонентов:

(вывод из консоли сокращен)

--- Тест 1: Sin и Cos

x	sin(x)	cos(x)
0,0	0,0	1,0
1,0	0,841471	0,540302
2,0	0,909297	-0,416147
3,0	0,141120	-0,989992
3,1	0,041581	-0,999135

--- Тест 2: Табулированные Sin и Cos

Сравнение оригинальных и табулированных функций:

x	sin(x)	tabSin(x)	cos(x)	tabCos(x)
0,0	0,0	0,0	1,0	1,0
0,5	0,479426	0,472070	0,877583	0,864608
1,0	0,841471	0,835841	0,540302	0,535972
1,5	0,997495	0,984808	0,070737	0,070437
2,0	0,909297	0,898147	-0,416147	-0,411747
3,0	0,141120	0,138735	-0,989992	-0,975537
3,1	0,041581	0,040753	-0,999135	-0,992814

--- Тест 3: $\sin^2(x) + \cos^2(x)$ с разным числом точек

Количество точек: 5

x	$\sin^2 + \cos^2$	Отклонение от 1
0,0	1,0	0,0
0,5	0,864487	0,135513
1,0	0,883675	0,116325
1,5	0,951957	0,048043
2,0	0,854819	0,145181
2,5	0,912382	0,087618
3,0	0,913432	0,086568
3,1	0,970621	0,029379

Максимальная ошибка: 0,146396

Количество точек: 50

x	$\sin^2 + \cos^2$	Отклонение от 1
0,0	1,0	0,0
0,5	0,999339	0,000661
1,0	0,999012	0,000988
1,5	0,999017	0,000983
2,0	0,999357	0,000643
2,5	0,999971	0,000029
3,0	0,999322	0,000678
3,1	0,999064	0,000936

Максимальная ошибка: 0,001026

Количество точек: 100

x	$\sin^2 + \cos^2$	Отклонение от 1
0,0	1,0	0,0
0,5	0,999814	0,000186
1,0	0,999748	0,000252
1,5	0,999802	0,000198
2,0	0,999975	0,000025
3,0	0,999750	0,000250
3,1	0,999784	0,000216

Максимальная ошибка: 0,000252

--- Тест 4: Write/Read экспоненты

Функция записана в файл: exp_tabulated.txt

Функция прочитана из файла

Сравнение исходной и считанной функции:

x	Исходная	Считанная	Разница
0	1,0	1,0	0,0
5	148,413159	148,413159	0,0
10	22026,465795	22026,465795	0,0

--- Тест 5: Output/Input логарифма

Функция записана в файл: log_tabulated.bin

Функция прочитана из файла

Сравнение исходной и считанной функции:

x	Исходная	Считанная	Разница
1	0,0	0,0	0,0
5	1,609438	1,609438	0,0
10	2,302585	2,302585	0,0

--- Тест 6: Сериализация

Сравнение (должно быть ? x):

x	Исходная		Считанная	Ожидаемое (x)
0	0,0	0,0	0	
5	5,0	5,0	5	
10	10,0	10,0	10	

Задание 9

Реализация сериализации:

Вариант 1: Serializable

```
public class LinkedListTabulatedFunction implements TabulatedFunction, Serializable {  
    // автоматическая сериализация  
}
```

```
public class ArrayTabulatedFunction implements TabulatedFunction, Serializable {  
    ...  
}
```

```
public class FunctionPoint implements Serializable{  
    ...  
}
```

Вариант 2: Externalizable

```
public class ArrayTabulatedFunction implements TabulatedFunction, Externalizable {  
    //конструктор без параметров  
    public ArrayTabulatedFunction() {  
        this.points = null;  
        this.pointsCount = 0;  
    }  
  
    // ручное описание записи  
    @Override public void writeExternal(ObjectOutput out) throws IOException {  
        out.writeInt(pointsCount);  
        for (int i = 0; i < pointsCount; i++) {  
            out.writeDouble(points[i].getX());  
            out.writeDouble(points[i].getY());  
        }  
    }  
  
    // ручное описание чтения  
    @Override public void readExternal(ObjectInput in) throws IOException, ClassNotFoundException {  
        pointsCount = in.readInt();  
        points = new FunctionPoint[pointsCount];  
        for (int i = 0; i < pointsCount; i++) {  
            double x = in.readDouble();  
            double y = in.readDouble();  
            points[i] = new FunctionPoint(x, y);  
        }  
    }  
}
```

Аналогично для LinkedList